SwarmSort: A Fair Evaluation of Particle Swarm Hyperparameter Optimization for YOLOv8 Waste Detection Across Studio, Real-World, and Indian Data

Anushka Gupta, Ayush Kumar, and Vaishnavi

***Abstract*—Automatic waste sorting requires detectors that can locate every item in an image, but a large part of the existing work still classifies single objects photographed against plain backgrounds. Moreover, studies that tune such models with metaheuristics seldom include a random-search baseline or report confidence intervals. In this paper, we examine whether particle swarm optimization (PSO) yields a better lightweight waste detector than random search or the library default settings when all three are given the same budget. Six training hyperparameters of YOLOv8n are selected in three ways: the defaults, the best of 32 random samples, and a swarm of 8 particles run for 4 rounds. Both searches use 32 short proxy trainings, and the proxy was verified beforehand against longer training (Spearman correlation 0.976, first generation). Each selected setting is then trained fully and tested once on held-out data, and the differences are assessed with paired-bootstrap 95% confidence intervals. The experiment was carried out in two generations: the first uses only studio images, while the second adds real-world photographs and an Indian street set that is kept for testing only. On studio data the three methods are tied, with a mean average precision at an intersection-over-union of 0.5 (mAP@50) of 0.676, 0.672 and 0.668 for the defaults, random search and PSO, respectively. When real-world training data are added, both searches outperform the defaults significantly on the mixed test set (by 0.017 and 0.020 mAP@50), but they do not differ from each other. On real-world photographs (0.298–0.310) no difference is significant, and on the Indian photographs all models reach only 0.023–0.027. On identical test images, adding real-world training data improved real-world mAP@50 from 0.038–0.096 to 0.338–0.353 and reduced studio mAP@50 by 0.074–0.105, all significantly. These results show that the choice of optimizer has little effect in comparison with the training data, and that tuning claims should be supported by an equal-budget random-search baseline and confidence intervals.**

***Index Terms*—Waste detection, particle swarm optimization, hyperparameter optimization, YOLOv8, random search, real-world generalization, India.**

I. Introduction

RECYCLING, composting and safe disposal of waste are possible only when the different materials are kept apart. In India, the Solid Waste Management Rules, 2016 make every waste generator responsible for segregating waste at source into bio-degradable, non-bio-degradable and domestic hazardous streams (Rule 4), and prescribe green, white and black bins for bio-degradable, recyclable and other waste (Rule 15(h)) [1]. A camera-based system that can identify the material of each item would help this segregation at a smart bin, on a sorting line or at a collection point. The use of artificial intelligence in solid waste management is by now well established [2], [3].

Most of the vision work on waste sorting, however, deals with a simpler problem than the one described above. A typical study takes a photograph of one object, usually on a plain background, and assigns it a class, as in TrashNet [4] and the classifiers built on it [5]. A heap of waste, a conveyor belt or a street contains many items at once. Some of them touch or hide each other, and every item has to be located as well as named, which makes it an object-detection problem. Majchrowska et al. [6], whose work was the starting point for ours, first localize litter with one network and then classify each detected crop with a second one. Multi-object waste detection therefore exists, but it remains a smaller body of work than single-object classification, and several detection studies still depend on single-object studio images. We refer to this as the task and scene gap (G1).

The second issue concerns hyperparameters. The accuracy of a detector depends on training settings such as the learning rate, momentum, weight decay and the strength of augmentation. The authors of TrashNet, for instance, attributed the 22% test accuracy of their convolutional neural network (CNN) to poorly chosen hyperparameters [4]. Metaheuristics have been used to tune CNNs, e.g., particle swarm optimization (PSO) [7] for a mammography classifier [8] and a beluga whale optimizer for an InceptionV3 waste classifier [9]. The evidence behind the reported improvements is nevertheless limited. In the studies that we reviewed, metaheuristic tuning in waste vision was applied only to classifiers, and we did not find any waste detector whose training hyperparameters were tuned by a swarm optimizer (G2). Furthermore, the tuned models were compared either with other metaheuristics or with no baseline at all, but never with random search [10] at an equal number of evaluations, and the test results were given as single values without confidence intervals (G3). Under these conditions it is not possible to say whether an improvement is due to the optimizer or simply to the fact that many settings were tried.

The data form the third issue. According to recent reviews, many waste-sorting studies work in simplified environments with artificially collected images [11], which do not resemble

The authors are with the Department of Computer Science and Engineering, Netaji Subhas University of Technology (NSUT), New Delhi, India.

real waste [3]. Accuracy also varies strongly from one dataset to another. The same YOLOv5x model, trained and tested separately on two datasets, obtained 84.9 AP_{50} on PlastOPol but only 63.3 on TACO [12]. An improvement obtained by tuning on studio images may therefore not carry over to street photographs. This matters for India, where segregation at source is compulsory. The closest available resource, the Dense Waste Segmentation Dataset (DWSD) [13], reports only pixel accuracy for segmentation, and we could not find an Indian real-world waste test set with published object-detection baselines (G4).

In this work, named SwarmSort, the tuning question is studied under controlled conditions. We train YOLOv8n [14], a nano-sized detector that can run on a smart bin or a mobile phone, and compare three ways of choosing six of its training hyperparameters, which we call arms: the library defaults (Arm A), the best of 32 random samples (Arm B) and our own implementation of PSO with 8 particles and 4 rounds (Arm C). Each search is allowed 32 short proxy trainings. The setting chosen by each arm is trained once for 100 epochs and tested once on a held-out split, and every difference is reported with a paired-bootstrap 95% confidence interval. The experiment was conducted in two generations. Generation 1 used studio-style images of six classes, and Generation 2 added real-world photographs, a seventh class (OTHER) and an Indian test set derived from DWSD. The following research questions are addressed:

RQ1: With an equal budget of 32 proxy trainings, does PSO find YOLOv8n training hyperparameters that perform better than those of random search and the default settings on a held-out test set? (G2, G3)

RQ2: Does a short proxy training (12 epochs on 40% of the data) rank hyperparameter settings in the same order as a longer training? (G3)

RQ3: Does the answer to RQ1 change when the data move from studio images to real-world and Indian scenes, and how large is the drop from studio to real-world images? (G1, G4)

The main contributions of this paper are summarized below.

- *A lightweight multi-object waste detector* (G1). YOLOv8n is trained on multi-object studio and real-world scenes, with six classes in Generation 1 and seven in Generation 2 (Sections III-B and III-C).
- *Swarm-based tuning of a waste detector* (G2). A PSO implemented by us searches the learning-rate schedule, momentum, weight decay and two augmentation settings (Sections III-D and III-E). To the best of our knowledge, the training hyperparameters of a waste detector have not been tuned with a swarm optimizer before.
- *An equal-budget protocol with three arms* (G3). The protocol includes a proxy-validity check whose pass threshold was decided beforehand (passed with Spearman $\rho = 0.976$, $n = 8$, Generation 1 only), a test split that is used only once, and paired-bootstrap 95% confidence intervals on all differences (Sections III-A, III-G and III-I).
- *Evaluation over two generations and three domains* (G4). Studio, real-world and Indian results are reported separately. These include detection results on DWSD, for which we are not aware of any published detection baseline. It should be noted that our DWSD boxes are derived from mask regions in which touching items are merged, and that the class mapping had to be inferred from the masks, so these numbers also reflect a mismatch in labeling. On the same test images, real-world training data increased real-world mAP@50 from 0.038–0.096 (Generation 1) to 0.338–0.353 (Generation 2), with a loss of 0.074–0.105 on studio images; all these changes are significant (Section IV-E).
- *Reporting of null results* (O5). PSO did not beat random search by any meaningful margin. Its only significant advantage, $+0.003$ mAP@50 on the Indian set, occurs where all scores are close to zero. Tuning gave no gain over the defaults in Generation 1 and a small gain in Generation 2 ($+0.017$ and $+0.020$ on the mixed test set), which did not extend to real-world or Indian photographs. We discuss the reasons in Section V, and we release the code, search logs, result files and trained models.

The rest of the paper is organized as follows. Section II reviews the related work and identifies the gaps G1–G4. Section III describes the datasets and methods. Section IV presents the results for RQ2, RQ1 and RQ3, in that order. Section V discusses the findings along with their implications and limitations, Section VI outlines future work, and Section VII concludes the paper.

II. Related Work and Research Gaps

In this section, we review the literature on vision-based waste recognition, on waste datasets from India and South Asia, and on hyperparameter optimization. The research gaps are stated at the end of the section.

A. Review Method

The literature search was carried out in October 2026. Crossref, arXiv, Europe PMC, PubMed Central, OpenAlex and publisher websites were searched with keywords related to waste reviews, datasets and detectors, to India and South Asia, and to metaheuristic and hyperparameter optimization. We also followed the citations of six seed papers [4]–[6], [8], [9], [15], all of which are discussed below. Vision studies on waste or litter and optimization studies on convolutional neural networks (CNNs) were included. Peer-reviewed papers published between 2016 and 2026 were preferred, although the classic method papers are older, and a few preprints, technical reports and workshop papers were retained where no peer-reviewed source covered the topic; their status is mentioned where they are cited. Fifteen studies were examined in detail and are compared in Table I, while the remaining works are cited for their stated scope. Since the review is targeted and not systematic, the statements made below about "the literature" apply only to the works we reviewed.

B. Vision-Based Waste Recognition: From Classification to Detection

A large share of waste-sorting research classifies images that contain a single object. The TrashNet dataset was released

by Yang and Thung [4] as part of a Stanford course report. Each of its 2,527 public images shows one item placed on a white posterboard and belongs to one of six classes. In that report, a support vector machine (SVM) with SIFT features achieved a test accuracy of 63%, whereas a CNN trained from scratch reached only 22%. The authors attributed the poor CNN result to unsuitable hyperparameters and left a more thorough hyperparameter search, as well as multi-object detection, for future work. Kaya [5] truncated pretrained Xception and InceptionResNetV2 networks and obtained 89.72% test accuracy with one fixed training setting, noting the plain backgrounds of TrashNet as a limitation. Sayed et al. [9] used an InceptionV3 tuned by a metaheuristic, which is discussed in Section II-D. In the smart bin of Alshammari et al. [15], items are routed by a MobileNetV3-based classifier running on a Raspberry Pi. TrashBox [16] carries the word "detection" in its title but is presented as a classification dataset of 17,785 images in seven classes. A single label per image is adequate when items are dropped one at a time into a chute. It is not adequate for a heap, a conveyor or a street.

The review papers come to a similar conclusion. Abdallah et al. [2] surveyed 85 studies on artificial intelligence in solid waste management published between 2004 and 2019. Lu and Chen [11] observed that many vision studies on waste sorting are preliminary, are conducted in simplified environments and use artificially collected data; they recommended real-world complexity and open datasets. Fang et al. [3] reported that most of the vision studies covered by them classify whole images by material, that detection studies are fewer and less accurate, and that laboratory pictures differ from real waste. The accuracies collected in their review range from 72.8% to 99.95%, but classification and detection results are mixed together, so the methods cannot be ranked from these numbers.

Datasets for detection and segmentation bring the problem closer to cluttered scenes. TACO [17], a crowd-sourced dataset described in a preprint, contained 1,500 photographs of litter in the wild with 4,784 annotations when it was released. ZeroWaste [18] was recorded on the sorting line of a recycling facility and contains deformable and translucent objects in heavy clutter, and WaRP [19] addresses hierarchical waste detection in images from a recycling plant. UAVVaste [20] consists of 772 low-altitude drone images with 3,716 annotations. Its authors pointed out that the large, near-ground objects of TACO are not suitable for aerial detection. PlastOPol [12] offers 2,418 crowd-sourced images of urban areas, beaches and forests with 5,300 annotations of a single "litter" class. The TrashCan technical report [21] provides boxes and masks for 7,212 deep-sea ROV images. FloW [22] covers floating waste in inland waters and is partly supported by radar data, while MJU-Waste [23] contains 2,475 RGB-D laboratory images of people holding waste, labeled with one "waste" class. Aerial, underwater, conveyor, hand-held and street settings thus all have their own datasets. Many of these have only one generic class, and their class definitions do not match one another.

The pipeline closest to ours is that of Majchrowska et al. [6], from which this work started. They merged public datasets into a one-class "detect-waste" set and a seven-category "classify-waste" set based on the sorting rules of Gdańsk. Because direct multi-class detection gave poor results, they localized litter with EfficientDet-D2 and classified each crop with EfficientNet-B2. Their hyperparameters were set manually, and one result per source dataset was reported on a held-out 20% described as a validation and testing set. Fulton et al. [24] compared YOLOv2, Tiny-YOLO, Faster R-CNN and SSD for underwater litter on desktop and embedded hardware. Melinte et al. [25] annotated TrashNet with boxes and reported a precision of 97.63% with SSD. Liao and Juang [26] ran YOLOv4-Tiny-3l at 22 frames per second on an Xavier NX to monitor coastal litter from a drone. In a workshop paper, Bartolo et al. [27] trained five detectors with privileged information on one drone dataset and tested them on two other datasets. Sánchez-Ferrer et al. [28] studied Mask R-CNN for marine debris when data are scarce and added synthetic scenes. In all these studies, the hyperparameters are treated merely as part of the experimental setup (Table I).

The detector used in SwarmSort belongs to the single-stage family that began with YOLO [29]. This family is frequently chosen in waste studies where speed is important [12], [20], [26], [30]. In the PlastOPol study, for example, YOLOv5x was the most accurate of seven detectors and YOLOv5s ran at up to 5.19 frames per second on a phone. We use YOLOv8n [14] initialized with COCO-pretrained weights [31] (Section III-C).

C. Waste Data From India and South Asia

Image data on waste from India are scattered, and most of them were not created for detection. The earliest work we came across is SpotGarbage [32] from IIT Ropar, a smartphone application that detects and coarsely segments garbage in geo-tagged photographs so that citizens can report it. Kumar et al. [30] motivated their study of YOLOv3 and YOLOv3-tiny with Indian waste statistics. They collected 6,437 images in six classes and reported 94.99% and 51.95% mAP on a split that served for both validation and testing. Their images, however, are mostly clean and show one or a few items each, and the place where they were taken is not mentioned. AquaVision [33], written mainly by Indian authors, detects waste in water bodies using a subset derived from TACO. TrashBox [16] from NIT Karnataka is a classification dataset, as mentioned earlier.

DWSD [13] comes closest to an Indian street benchmark. It contains 784 images taken with phone cameras in streets, parks, lawns and garbage areas around Kolkata (Jadavpur, Garia) and Berhampore in West Bengal, with pixel-level masks for 14 waste classes. The baselines in the paper report only pixel accuracy (89.39% for DeepLabv3+), and no detection results are given. We also noticed that the paper is not consistent about the name of the dataset and the number of images. From the rest of South Asia, UGV-NBWASTE [34] provides 3,600 images of eight types of non-biodegradable waste from Barisal, Bangladesh. It has oriented boxes, but only about 1.1 objects per image and no baseline models. The pLitterStreet preprint [35] maps plastic litter at street level in Thailand, Vietnam and Sri Lanka using cameras mounted on vehicles. Fang et al. [3] also mention the scarcity of data in developing countries as one of the limitations of the field.

D. Hyperparameter Optimization and Swarm Intelligence

Particle swarm optimization (PSO) was proposed by Kennedy and Eberhart [7]. A population of candidate solutions, called particles, moves through the search space, and the velocity of each particle is a combination of its previous velocity and the attraction toward its own best position and the best position of the swarm. Shi and Eberhart [36] later added the inertia weight, which scales the previous velocity. Clerc and Kennedy [37] analyzed the particle trajectories and derived coefficients that control convergence, which led to constriction-factor PSO. PSO does not require gradients and evaluates a fixed number of candidates in every round. This makes it convenient when each evaluation is expensive, as in the case of training a network. The settings used in our work are given in Section III-E.

Random search [10] is the obvious reference for any guided search method. It samples the settings independently and can be parallelized without restriction. If an optimizer cannot do better than random search with the same number of evaluations, there is no proof that its guidance is useful. Multi-fidelity methods take a different route and reduce the cost of each evaluation. Hyperband [38], for example, accelerates random search by allocating more resources to promising configurations and stopping the weak ones early. SwarmSort uses a simpler approach with one fixed low fidelity, namely a short proxy training (Section III-G).

Metaheuristics have been applied to tune CNNs in several domains, but we found the supporting evidence to be thin. Aguerchi et al. [8] used PSO to select three architecture hyperparameters (number of filters, kernel size and stride) of a small CNN for mammogram masses, while the training hyperparameters were kept fixed. Their swarm had 4 particles and ran for 3 iterations, which amounts to roughly 12 network evaluations. No baseline (default setting, random search or another optimizer) was included and only a single result was reported, even though the authors themselves remark that a metaheuristic requires repeated runs and statistical analysis. The most complete comparison in the waste domain that we found is by Sayed et al. [9]. A multi-objective beluga whale optimizer (MBWO) tunes the learning rate, batch size and dropout period of InceptionV3 on TrashNet with a population of 30, 20 iterations and 15 epochs per evaluation. Four other metaheuristics are run with the same population, iterations, bounds and fitness function, and an untuned model serves as reference (90.76% accuracy). A Wilcoxon rank-sum test on the best search fitness over independent runs shows that MBWO is significantly better than three of the competitors but not the fourth (EO). On the other hand, the number of runs is not stated, random search is not included, the test metrics are single values without intervals, and the headline accuracy is 97.75% in the abstract but 97.57% in the results table. Mao et al. [39] applied a genetic algorithm to a CNN that classifies recycling waste.

E. Comparative Analysis and Research Gaps

Table I compares the 15 studies that we examined in detail with respect to the points that determine whether a tuning claim is reliable. Only two of them tune hyperparameters with a metaheuristic. Both are classifiers, and only one compares tuning methods. None of the studies reports a confidence interval on a test metric, and only Bartolo et al. [27] test on public datasets that were not seen during training. Based on these observations and on Sections II-B–II-D, we identify four gaps. For each gap, the evidence that goes against it is also mentioned.

Gap 1 (task and scene). The classifiers discussed in Section II-B [4], [5], [9], [15], [16] give one label per image, and those based on TrashNet see one object on a plain background. Fang et al. [3] found whole-image classification to be the most common approach. Even among the detection studies, some depend on data that have nearly one object per image [25], [30], [34]. At the same time, multi-object waste detection is clearly an active line of work, mostly in computer-vision, robotics and sensing venues, as the datasets and the pipeline in Section II-B show [6], [12], [17]–[22]. As our review is not systematic, we do not attempt to quantify how much smaller this line is. The gap can be stated as follows: *in the reviewed literature, waste recognition for sorting is dominated by single-label classification of one object on a plain background; multi-object detection in cluttered scenes is a smaller strand, and several detection studies still rely on single-object studio images.*

Gap 2 (optimization). In waste vision, metaheuristic tuning has been applied to classifiers [9], [39], and PSO tuning of CNNs is found in other domains [8]. None of the waste detection or segmentation studies in Table I describes a search over training hyperparameters. The only search that is described concerns post-processing parameters [23], and Fang et al. [3] mention PSO and genetic algorithms only in connection with routing and prediction. The gap is therefore limited to the combination of metaheuristic tuning with waste detectors. Two detection papers mention tuning without describing it [26], [27], so a search in those cases cannot be excluded, and for the works that were not examined in detail the training setup could not be verified. The gap is: *in the reviewed literature, no waste detector had its training hyperparameters tuned by a swarm optimizer; detection studies used default or hand-set values, or a tuning step that is not described.*

Gap 3 (evidence). When tuning is evaluated, the baselines are weak. Aguerchi et al. [8] have none, and Sayed et al. [9] compare only with other metaheuristics; in both cases the test metrics are single numbers (Section II-D). Among the detection and segmentation papers in Table I, none reports run-to-run variation, confidence intervals or significance tests for its accuracy, and Bartolo et al. [27] state that statistical analysis was not applicable. It must be acknowledged that Sayed et al. do compare optimizers with equal population and iterations, include an untuned reference and apply a significance test to the search fitness. We therefore do not claim that optimizers are never compared or that significance is never tested. The gap is: *metaheuristic tuning in this area is compared against other metaheuristics or against nothing; we found no comparison with random search [10] at an equal evaluation budget, and test results are reported as single numbers without confidence intervals.* SwarmSort reduces this gap but does not close it

TABLE I
COMPARISON OF EXISTING STUDIES WITH SWARMSORT

Study	Task	Scenes	Hyperparameters set by	Tuning baselines	Runs / CIs	Cross-domain test	India data
Yang & Thung [4]	Cls	Studio	Manual	—	1 / no	No	No
Kaya [5]	Cls	TrashNet	Fixed	—	1 / no	No	No
Sayed et al. [9]	Cls	TrashNet	MBWO	4 metaheur. (equal), untuned; no RS	n.r. + Wilcoxon (fitness) / no	No	No
Aguerchi et al. [8]	Cls	Clinical	PSO (architecture)	None	1 / no	No	—
Majchrowska et al. [6]	Det + Cls	Real-world	Manual	None	1 / no	No	No
Hong et al. [21]	Det / Seg	Underwater	Fixed	None	1 / no	No	No
Kraft et al. [20]	Det	Aerial	Not described	None	1 / no	No	No
Córdova et al. [12]	Det	Real-world	Published defaults	None	No SDs / no	Per dataset	No
Wang et al. [23]	Seg	Lab + TACO	Fixed; CRF grid	None	1 / no	No	No
Fulton et al. [24]	Det	Underwater	Community	None	1 / no	Shifted footage	No
Melinte et al. [25]	Det	TrashNet	Manual + LR test	None	1 / no	Lab-made set	No
Kumar et al. [30]	Det	Mostly clean	Fixed	None	1 / test = val	No	Unstated
Liao & Juang [26]	Det	Aerial	"Tuned" (n.r.)	None	No	Qualitative	No
Bartolo et al. [27]	Det	Aerial	Manual	None	1 / no	Yes	No
Ali et al. [13]	Seg	Real-world	n.r.	—	1 / no	No	Yes
SwarmSort (ours)	**Det**	**Studio + real-world**	**PSO vs RS vs defaults**	**Equal: 32 proxy trainings each**	**1 seed / bootstrap 95% CIs**	**Yes (3 domains)**	**DWSD (test)**

Datasets: TrashNet [4] (single-object studio images), TrashCan [21], UAVVaste [20], PlastOPol [12], MJU-Waste [23] and DWSD [13] (a dataset paper). Cls = classification; Det = detection (one class in [6], [12]); Seg = segmentation; MBWO = multi-objective beluga whale optimizer; metaheur. (equal) = other metaheuristics with the same population size and iterations; RS = random search; CRF grid = grid search for the conditional random field only; LR test = learning-rate range test; SD = standard deviation; CI = confidence interval; n.r. = not reported; — = not applicable. "1 / no" = one reported run per configuration and no confidence intervals. "Fixed" = one setting used without a described selection procedure. "test = val" = the split used for validation also served as the test split. Per dataset = two datasets, each trained and tested separately; Qualitative = new sites described only qualitatively; Unstated = Indian authors, image location not stated. SwarmSort: 6 classes in Generation 1 and 7 in Generation 2; one training seed per arm; paired-bootstrap CIs; studio, real-world and Indian test sets. The review of Fang et al. [3] and the UGV-NBWASTE dataset [34], which reports no model, are not listed.

completely. Since we use one training seed per arm and one search seed per method, our intervals account only for the sampling of the test set (Section V-D).

Gap 4 (generalization). The TrashNet-based classifiers are never evaluated outside the studio, two reviews point to simplified, laboratory-like data [3], [11], and accuracy depends strongly on the dataset, as the PlastOPol example in Section I shows [12]. Most detection studies test within a single dataset or describe the degradation at new sites only qualitatively (Table I), and the Indian work in Section II-C does not provide a detection benchmark. Cross-domain tests are not entirely absent. Bartolo et al. [27] test on datasets not seen in training, Fulton et al. [24] on footage that differs in time and place, and Sánchez-Ferrer et al. [28] add synthetic scenes to scarce real data. South Asian detection data are available in UGV-NBWASTE [34], although without baselines, and in the pLitterStreet preprint [35]. The gap is: *studio-to-real-world and cross-dataset evaluation is rare and recent, and we found no Indian real-world waste test set with published object-detection baselines.* SwarmSort reports studio, real-world and DWSD results separately and compares studio-only training with mixed training on the same test images (Section IV-E). As already mentioned, the DWSD boxes are obtained from mask regions that merge touching items and the class mapping is inferred from the masks, so the DWSD results are affected by a labeling mismatch as well.

Observation O5 (reporting of positive results). The applied papers that we read present their methods as improvements. Sayed et al. [9], for example, describe their optimizer as the best although its advantage over EO is not significant. Some negative findings are reported in part: a CNN that performs far below an SVM [4], direct multi-class detection being abandoned because of poor results [6], synthetic data that help with data scarcity only to a limited extent [28], and lower performance on shifted data [24], [26]. We did not, however, find a paper reporting that a tuning method failed to beat a simple baseline. Since the absence of such papers in a targeted sample proves little, we list this as an observation and not as a gap. In SwarmSort the null and negative results are reported and analyzed (Sections IV and V). Table II shows how each gap is addressed in this work.

TABLE II
RESEARCH GAPS AND HOW SWARMSORT ADDRESSES THEM

Gap	How SwarmSort addresses it	Where
G1: task and scene	Multi-object detection with YOLOv8n (6 or 7 classes) on studio and real-world scenes	Sec. III-B, III-C
G2: optimization	Our PSO implementation tunes 6 training hyperparameters of a waste detector	Sec. III-D, III-E
G3: evidence	Defaults, random search and PSO at 32 proxy trainings each; proxy check with a threshold fixed in advance (Generation 1 only); test split used once; paired-bootstrap 95% CIs	Sec. III-A, III-G, III-I, IV-C, IV-D
G4: generalization	Real-world training data in Generation 2; studio, real-world and India (DWSD, test only, boxes from masks) results reported separately; both generations on the same test images	Sec. III-B, IV-E
O5: reporting	Null and negative results reported and explained, with limitations	Sec. IV, V

III. Materials and Methods

The design of the study follows from the gaps listed in Section II-E. We use a multi-object detector and not a classifier (G1), apply a swarm optimizer to the training hyperparameters of this detector (G2), compare it with random search and the defaults at an equal budget using confidence intervals on a test split that is used once (G3), and keep separate studio, real-world and Indian test sets (G4).

A. Experimental Design

The training hyperparameters of one detector are selected by three arms, as shown in Fig. 1. Arm A (defaults) takes the Ultralytics default values without any search and acts as the reference that tuning has to beat. Arm B (random search) selects the best of 32 random samples, and Arm C (PSO) selects the best setting found by a swarm of 8 particles in 4 rounds. The two search arms evaluate their candidates with the same proxy training (Section III-G). The budget is therefore measured in proxy trainings and not in wall-clock time, since the latter varied between the methods and between cloud sessions (Section IV-F). The setting selected by each arm is trained once using an identical recipe (Section III-H) and evaluated once on a held-out test split (Section III-I).

The whole experiment was performed twice. Generation 1 is based on a single studio-style dataset with six classes and an image size of 416 px. Generation 2 adds real-world training photographs, a seventh class (OTHER), an image size of 640 px and an Indian set used only for testing. The search space, PSO settings, seeds, training recipes and evaluation code are common to both generations, and both searches were run again from the beginning for Generation 2. Because the classes, image size and training data are different, the scores of the two generations cannot be treated as one benchmark. A separate comparison of the generations is described in Section III-I.

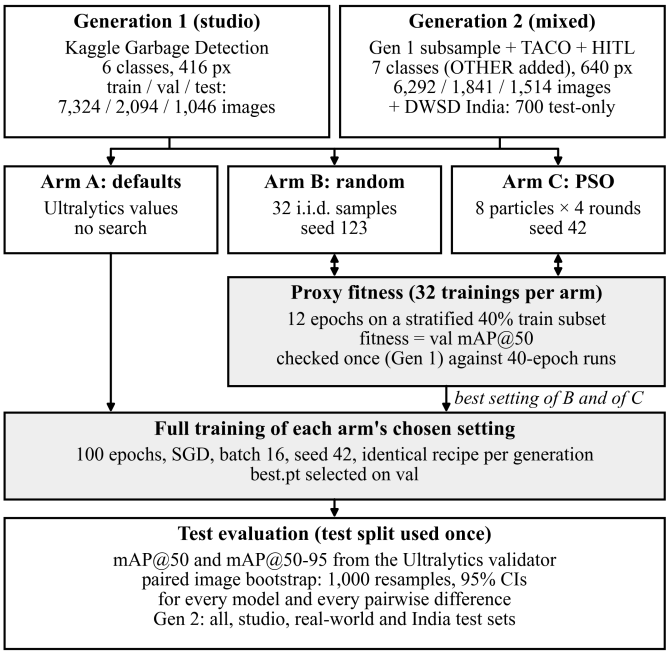


Fig. 1. Experimental pipeline used in both generations. Arms B and C are each allowed 32 proxy trainings, while Arm A uses the library defaults without any search. The setting chosen by each arm is trained once for 100 epochs and tested once.

B. Datasets

All data sources are listed in Table III together with their licences and split sizes.

Generation 1 data. We used the Kaggle dataset "Garbage Detection – 6 Waste Categories" [40], which originates from a Roboflow project. It contains studio-style images with YOLO-format boxes for six classes: BIODEGRADABLE, CARDBOARD, GLASS, METAL, PAPER and PLASTIC. The dataset is imbalanced. In the training split, 36.45% of the boxes occupy less than 1% of the image area, and BIODEGRADABLE accounts for 61.92% of the boxes whereas PAPER accounts for only 5.61%. The folders supplied with the dataset are divided by class; the validation folder, for instance, has 13,632 BIODEGRADABLE boxes but only 33 PAPER boxes, and the test folder has no GLASS box at all. For this reason we pooled all the images and split them again in a 70/20/10 ratio, stratified by source class (seed 42), ensuring at least 30 boxes of each class in the validation and test splits. Near-duplicate images were always kept in the same split. Two images were treated as copies if their files were identical (MD5), if they came from the same Roboflow source image, if their 256-bit difference hashes differed by at most 10 bits, or if the hashes differed by at most 40 bits and the Pearson correlation between their 64×64 grayscale thumbnails was 0.90 or higher. In this way 47 groups comprising 133 images were found. The label checks revealed 17 zero-size boxes, and no image was corrupt or unlabeled.

Generation 2 training data. Generation 2 has seven classes, i.e., the six classes of Generation 1 with the same indices and an additional class OTHER. From the Generation 1 splits we drew a subsample stratified by dominant class (seed 42) with 3,000 training and 900 validation images. All test images were retained, so the studio test set of Generation 2 is the same as the test set of Generation 1. TACO [17] contributes crowd-sourced photographs of litter in the wild. All of its 60 categories were mapped to our classes (drink cans to METAL and cigarettes to OTHER, for example), which gave 1,260 OTHER boxes but only 8 BIODEGRADABLE boxes. The HITL Recycling dataset [41] contributes street photographs, nearly all of which have a single litter box. Its objects were assigned a class from the object tag when this was decisive (food waste $\rightarrow$ BIODEGRADABLE), otherwise from the material tag (aluminum $\rightarrow$ METAL), and otherwise OTHER; 100 objects had no material tag. TACO and HITL were split 70/20/10 using the same duplicate rules, and their images were stored with the longer side limited to 640 px. The merged data contain 962 duplicate groups, and none of them extends over two sources or two splits. Within the real-world sources there are 918 byte-identical pairs. These do not cause leakage, but they give extra weight to some photographs. In the merged training set, BIODEGRADABLE has 51.96% of the boxes and OTHER has 3.89%. The studio images contain

no OTHER box, and the real-world test set contains only one BIODEGRADABLE box.

Indian test set. The Dense Waste Segmentation Dataset (DWSD) [13] consists of 784 phone photographs of streets, parks and garbage areas in Kolkata and Berhampore, West Bengal, annotated with 448×448 semantic masks. This dataset was used strictly for testing and played no role in training, search or model selection. To obtain boxes, every 8-connected region of a single mask value with at least 50 pixels was converted into one box. As a consequence, items of the same class that touch each other are merged into one large box. The class mapping had to be inferred by us, because the published table of mask values does not agree with what the regions actually show (the value that is listed as background covers polythene bags in 610 masks). We assigned a name to each value after inspecting its regions, and six of the fifteen values remain uncertain. Eighty-four images that yielded no boxes were excluded since some of them show waste that is not annotated. The remaining set has 3,040 PLASTIC, 1,230 PAPER, 124 CARDBOARD, 90 OTHER and 18 METAL boxes. An MD5 comparison with the 9,647 images of the main build did not find any identical file.

C. Detector

All three arms train YOLOv8n, the nano variant of the single-stage YOLOv8 detectors [14], [29], using Ultralytics version 8.4.169. Training starts from COCO-pretrained weights [31], and the class outputs are resized to six or seven classes. The nano model was chosen for two reasons: it is suitable for a smart bin or a phone with limited computing power, and it makes 32 proxy trainings per arm feasible on a single cloud GPU.

D. Search Space

Both search arms work on the same six dimensions, which are listed in Table IV. These are the initial learning rate `lr0`, the final learning rate expressed as a fraction of the initial one (`lrf`), the SGD `momentum`, the `weight_decay`, the probability of mosaic augmentation (`mosaic`) and the gain of random rescaling (`scale`). The optimizers operate in what we call search units. Since `lr0` and `weight_decay` cover two orders of magnitude, they are represented by their base-10 exponents, so that a particle at -3.2 corresponds to $\texttt{lr0} = 10^{-3.2}$. The other four dimensions are represented by their values. After decoding, the values are rounded to six significant digits. It may be noted that three of the default values lie on the boundary of the search space (`lrf` on the lower bound, `lr0` and `mosaic` on the upper bounds), which means that the searches can reach them only at the edge. All remaining training hyperparameters were left at the library defaults in every arm.

E. Particle Swarm Optimization

Arm C employs global-best PSO [7] with an inertia weight [36]. We implemented it ourselves in NumPy, and it maximizes the proxy fitness. Let $\mathbf{x}_i \in \mathbb{R}^6$ be the position of particle i in search units, $\mathbf{v}_i$ its velocity, $\mathbf{p}_i$ the best position it has visited and $\mathbf{g}$ the best position of the swarm. The velocity is updated as

$$\mathbf{v}_i \leftarrow w_t\,\mathbf{v}_i + c_1\,\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i) + c_2\,\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i), \qquad (1)$$

where $\odot$ denotes the element-wise product and $\mathbf{r}_1$, $\mathbf{r}_2$ are drawn afresh at every step (Table IV). Each velocity component $v_{i,d}$ is then limited to $[-v_{\max,d},\, v_{\max,d}]$ with $v_{\max,d} = 0.2\,(u_d - l_d)$, where l_d and u_d are the lower and upper bounds of dimension d. The position is updated element-wise as

$$\mathbf{x}_i \leftarrow \min\big(\max(\mathbf{x}_i + \mathbf{v}_i,\ \mathbf{l}),\ \mathbf{u}\big). \qquad (2)$$

If a dimension is clipped, its velocity is set to zero so that the particle stays at the boundary. The random initial swarm is evaluated in round 1, and each of the $T - 1 = 3$ update steps is followed by another round. Hence $T = 4$ rounds consume exactly 32 evaluations. The inertia weight decreases linearly according to

$$w_t = w_{\text{start}} - (w_{\text{start}} - w_{\text{end}})\,\frac{t-1}{T-2}, \qquad t = 1, \ldots, T-1, \qquad (3)$$

with $w_{\text{start}} = 0.9$ and $w_{\text{end}} = 0.4$. The idea of an inertia weight is taken from [36], while the linear schedule over three steps is our own choice, made so that the swarm explores in the beginning and refines toward the end. A personal best is replaced only when a strictly higher fitness is obtained. All random numbers come from one seeded generator. An interrupted search can therefore be resumed exactly by replaying the seed together with the logged fitness values. If a training had failed (non-finite mAP@50), it would have received a fitness of 0 and would never have been selected, but this did not happen in any of the four searches.

To verify the implementation, the optimizer was run on two standard test functions over $[-5.12, 5.12]^6$ with 20 particles, 50 rounds and 10 seeds. The median best value was 1.9×10^{-5} for the Sphere function and 6.96 for the multimodal Rastrigin function, both of which have a minimum of 0. The optimizer thus converges on a smooth bowl and continues to improve in the presence of many local minima. This test only confirms that the code is correct and says nothing about tuning performance.

F. Random Search and Defaults

Arm B samples 32 points uniformly in search units, which is log-uniform for `lr0` and `weight_decay`, and evaluates them with the same proxy and the same script as the swarm. Random search [10] is included because a structured optimizer is expected to beat it when the budget is the same. The winner of each search arm is the candidate with the highest proxy fitness, and the first one is taken in case of a tie. Arm A uses the default values given in Table IV.

G. Proxy Fitness and Its Validation

One full training takes more than an hour on our GPU. The searches therefore evaluate each candidate with a proxy training of 12 epochs on a stratified 40% subset of the training data. This subset has 2,930 of the 7,324 images in Generation 1

TABLE III
DATASETS. COUNTS ARE IMAGES, WITH BOXES IN PARENTHESES. "CLASSES" IS THE NUMBER OF TARGET CLASSES THAT OCCUR IN THE SOURCE. THE GENERATION 2 STUDIO TEST SET EQUALS THE GENERATION 1 TEST SET; THE GENERATION 2 REAL-WORLD TEST SET IS THE UNION OF THE TACO AND HITL TEST IMAGES (468 IMAGES, 779 BOXES).

Generation	Source	Domain	Licence	Stored size	Classes	Train	Validation	Test
Generation 1	Garbage Detection (Kaggle)	studio	CC BY 4.0	416×416	6	7,324 (51,756)	2,094 (14,904)	1,046 (7,413)
Generation 2	Generation 1 subsample	studio	CC BY 4.0	416×416	6	3,000 (22,206)	900 (6,904)	1,046 (7,413)
	TACO [17]	real-world	CC BY 4.0	long side ≤ 640	7	1,050 (3,293)	301 (1,031)	149 (460)
	HITL Recycling	real-world	CC0	long side ≤ 640	7	2,242 (2,246)	640 (641)	319 (319)
	Total (7 classes)	mixed			7	6,292 (27,745)	1,841 (8,576)	1,514 (8,192)
	DWSD [13]	India	CC BY 4.0	448×448	5	–	–	700 (4,502)

TABLE IV
SEARCH SPACE AND SEARCH SETTINGS (SHARED BY BOTH GENERATIONS)

Hyperparameter	Range	Scale	Default (Arm A)
`lr0`	10^{-4} to 10^{-2}	$\log_{10}$	0.01
`lrf`	0.01 to 0.2	linear	0.01
`momentum`	0.60 to 0.98	linear	0.937
`weight_decay`	10^{-5} to 10^{-3}	$\log_{10}$	0.0005
`mosaic`	0.0 to 1.0	linear	1.0
`scale`	0.2 to 0.8	linear	0.5
Setting	**Value**		
PSO budget	8 particles $\times$ 4 rounds $=$ 32 evaluations		
Inertia w_t	0.9, 0.65, 0.4 (linear over 3 update steps)		
c_1, c_2	1.5, 1.5		
$\mathbf{r}_1$, $\mathbf{r}_2$	$U(0,1)$, fresh per particle and dimension		
Velocity limit	$\pm 20\%$ of each range		
Initialization	positions uniform in bounds; velocities uniform in $\pm 10\%$ of each range		
Boundaries	position clipped; velocity of a clipped dimension set to 0		
PSO seed	42		
Random search	32 i.i.d. uniform samples in search units; seed 123		
Fitness	validation mAP@50 after a 12-epoch proxy training on a stratified 40% training subset		

(stratified by source class) and 40% of the 6,292 images in Generation 2 (stratified by source and dominant class). It is drawn once with seed 42 and saved as a list of images, so all candidates are trained on exactly the same images. The fitness is the validation mAP@50 of the weights after the last epoch. The proxy uses the recipe of the full training (Section III-H) with one adjustment: the no-mosaic period and the warm-up retain their proportion of the run, 10% and 3% respectively (`close_mosaic` of 1 epoch and a warm-up of 0.36 epochs). This keeps the effect of the searched `mosaic` value consistent with the full run.

Before starting the Generation 1 searches, we checked whether the proxy ranks the settings in the same way as a longer training does. Seven configurations were generated by Latin hypercube sampling [42] in search units (SciPy, "random-cd", seed 42). These seven and the defaults were each trained with the proxy and also with a "medium" run of 40 epochs on the full training split (`close_mosaic` 4, warm-up 1.2 epochs), both at 416 px. The Spearman rank correlation [43] of the eight pairs of scores was used as the statistic. The pass threshold of $\rho \geq 0.6$ and the fallback option of a 20-epoch proxy were both decided before the check was run. The outcome is given in Section IV-A.

This check was not repeated for Generation 2. The decision was recorded before the Generation 2 searches and was based on the Generation 1 result and the unchanged recipe. The validity of the proxy for the new data, classes and image size is thus an assumption and has not been tested. Further, the check involves only eight configurations and a reference of 40 epochs instead of 100, and the proxy scores the last-epoch weights whereas the final models use `best.pt` (Section V-D).

H. Full Training

The setting chosen by each arm was trained once for 100 epochs on the full training split, starting from the COCO-pretrained weights. The recipe was the same for all three arms. SGD was specified explicitly, because the default `optimizer=auto` would have ignored the searched values of `lr0` and `momentum`; each run logs the optimizer and learning rate actually used. The batch size was 16, the warm-up lasted 3 epochs, mosaic was switched off in the last 10 epochs, and the image size was 416 px for Generation 1 and 640 px for Generation 2. The checkpoint having the best validation fitness (`best.pt`) was taken for testing. All other arguments were left at their defaults.

Seed 42 and the deterministic mode were used for every run. During this work we found that a module which Ultralytics imports lazily in the first training of a process draws random numbers after the seed has been set. As a result, the first run of a process differed from the later ones. Importing this module before seeding solved the problem. All trainings were carried out on a single NVIDIA Tesla T4 GPU on Kaggle. Only one training seed was used per arm, so the variance between runs has not been measured (Section V-D).

Two runs of Generation 2 stopped responding, most probably because the data mount of the session froze. Arm C was resumed from its checkpoint of epoch 55 after every saved training argument had been compared with the plan. The resumed run continues with the same weights, optimizer state and epoch counter, but it is not bit-identical to a run without interruption. The random search had completed 11 of its 32 evaluations when it stopped, and it was continued by replaying its log, which left the completed evaluations unaffected.

I. Evaluation Protocol

Every test split was evaluated only once, after all training had finished. The validation split was used for all intermediate

evaluations. In Generation 2, a single run evaluated the three models on the complete test split (“all”, 1,514 images) and separately on the studio, real-world and India sets. Before evaluation, each checkpoint was checked against the record of its arm (classes, epochs, image size and the six hyperparameters).

The Ultralytics validator was run at the training image size of each model with a batch size of 32, rectangular batching, a confidence threshold of 0.001, an NMS IoU of 0.7 and 32-bit floats. We report mAP@50, mAP@50–95 (IoU from 0.50 to 0.95 in steps of 0.05), precision and recall at the confidence that maximizes F1, and AP@50 per class. As a consistency check, both mAP values were computed again from the stored per-image statistics and were required to match the validator within 10^{-4}. They matched exactly in all cases.

Uncertainty is estimated with a paired bootstrap over images [44]. The test images are resampled 1,000 times (seed 0), and the same indices are used for every model. We report 95% percentile intervals for each model and for each pairwise difference, and a difference is called significant only when its interval does not contain 0. A separate bootstrap is run for each Generation 2 test set. We did not compute p-values or apply corrections for multiple comparisons, and per-class AP is reported without intervals. The intervals reflect the sampling of test images only and do not include the randomness of training or search.

For comparing the two generations on the same images, all six final models (three arms from each generation) were evaluated on the studio and real-world test sets, each at its own training image size, using only the six shared classes. For this purpose the OTHER labels were removed (images that became empty were kept), and the OTHER predictions of Generation 2 were discarded during non-maximum suppression. Each test set again has its own bootstrap. These results are presented in Section IV-E and Table VIII.

Inference speed is reported as the median batch-1 latency over 100 test images, measured after 10 warm-up predictions at a confidence of 0.25 and at the training image size. The measurement was made on a laptop CPU (Intel Core Ultra 5 125H) for Generation 1 and on the T4 GPU for Generation 2. The latencies can therefore be compared only within a generation.

IV. Results

The results are presented in the following order: validity of the proxy (RQ2), behavior of the two searches and the test comparison of the three arms (RQ1), and the change from studio to real-world and Indian photographs (RQ3). Their interpretation is taken up in Section V.

A. Proxy Validity

The proxy passed the check that had been defined before the Generation 1 searches (Section III-G). The Spearman rank correlation between the proxy scores and the 40-epoch medium scores is $\rho = 0.976$ ($n = 8$), which is much higher than the pass threshold of 0.6 (Table V and Fig. 2). The two rankings are identical except that the neighboring settings lhs3 and lhs5 are interchanged. The same four settings are at the top of both rankings, and in both the defaults come second after lhs4. However, the medium scores of these four settings are within 0.026 of one another. The proxy is therefore much better at separating poor settings from good ones than at ordering the good ones among themselves.

TABLE V
Proxy-Validity Check (Generation 1, Validation mAP@50)

Setting	Proxy	Rank	Medium	Rank
lhs1	0.08687	1	0.28423	1
lhs2	0.23077	2	0.49451	2
lhs3	0.29495	3	0.56712	4
lhs5	0.30186	4	0.55195	3
lhs7	0.37560	5	0.61246	5
lhs6	0.44285	6	0.62161	6
Defaults	0.47648	7	0.62884	7
lhs4	0.49547	8	0.63839	8
Spearman $\rho = 0.976$, $n = 8$; pass threshold $\rho \geq 0.6$				

Proxy: 12 epochs on the stratified 40% training subset. Medium: 40 epochs on the full training split. Both at 416 px; rank 1 is the lowest score. lhs1–lhs7 are the Latin hypercube settings.

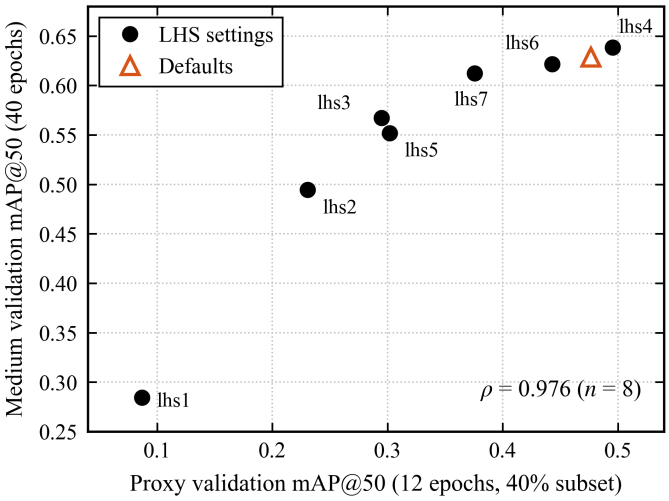


Fig. 2. Proxy versus medium validation mAP@50 of the eight settings used in the Generation 1 proxy-validity check. The two rankings agree except for the neighboring pair lhs3 and lhs5.

A second, indirect check is possible because some configurations were evaluated in both generations. As the seeds and the search space were the same, random search sampled the same 32 configurations in Generation 1 and Generation 2. In PSO, the positions depend on the fitness only through the personal and global bests. These coincided long enough for the swarm to visit identical positions in rounds 1–3 (24 evaluations), but not in round 4. For these repeated configurations, the Spearman correlation between the proxy scores of the two generations is 0.991 for random search and 0.892 for PSO, computed from the four search logs. It should be kept in mind that this compares one proxy with another proxy and not a proxy with longer training. It shows that the two generations order the settings similarly, but it is not a substitute for a proxy-validity check on Generation 2.

B. Search Behavior

Fig. 3 plots all proxy evaluations along with the best score obtained so far. In Generation 1, random search reached its best setting (proxy mAP@50 of 0.4995) at the second evaluation itself and did not improve afterwards. PSO improved gradually, from 0.4894 after round 1 to 0.4942 after round 3 and 0.4953 in round 4 (evaluation 27). The swarm converged as expected: the mean score per round increased from 0.393 to 0.423, 0.450 and 0.482, while the standard deviation within a round decreased from 0.122 to 0.088, 0.047 and 0.011. The defaults obtained 0.4765 on the same proxy, which is higher than 28 of the 32 evaluations of random search and 21 of the 32 evaluations of PSO. The two winners are separated by only 0.0042, and lhs4 from the proxy check (0.4955) falls between them. Interestingly, the four search winners of the two generations all have an initial learning rate (0.0025 to 0.0070) below the default value of 0.01.

In Generation 2, random search again found its best value early (0.3340 at evaluation 3). The best value of PSO, 0.3290, belonged to the last particle of round 1 and was not improved in the later rounds, whose best values were 0.3176, 0.3237 and 0.3279. The swarm nevertheless converged, with the round means rising from 0.267 to 0.286, 0.300 and 0.319 and the within-round standard deviation falling from 0.082 to 0.006. In both generations the mean proxy score of PSO over its 32 evaluations is higher than that of random search (0.437 versus 0.333 in Generation 1 and 0.293 versus 0.226 in Generation 2). This is because the swarm places its later evaluations close to good points; it does not mean that better points were found. No proxy run of the defaults was made in Generation 2, so their position on the Generation 2 proxy is not known.

C. Generation 1 Test Results (Studio Data)

The test results of the three Generation 1 models on the 1,046 studio test images are given in Table VI. (The evaluator counts 7,415 boxes, whereas Table III lists 7,413 because 2 zero-size boxes are omitted there.) The mAP@50 is 0.676 for Arm A, 0.672 for Arm B and 0.668 for Arm C. For both metrics, the 95% interval of every pairwise difference includes 0, and the largest difference is C$-$A with -0.008 [-0.021, $+0.007$]. The three arms are thus statistically tied on studio data. Going by the point estimates alone, the defaults are in fact first and PSO is last. Precision and recall are almost the same for the three arms. The validation scores agree with this picture. The defaults and PSO both reached 0.657, and random search, whose setting had the highest proxy score in Generation 1, reached the lowest value of 0.643.

Fig. 4 shows the AP@50 for each class (without confidence intervals, see Section III-I). On five of the six classes the arms are within 0.02 of each other. PLASTIC is the exception: PSO obtains 0.638, compared with 0.699 for Arm A and 0.693 for Arm B. PSO has the highest value on BIODEGRADABLE, GLASS and METAL. Since the three networks have the same architecture, the higher CPU latency of Arm B in Table VI is most likely due to other load on the laptop at the time of measurement.

D. Generation 2 Test Results (Mixed Data)

Table VII lists the Generation 2 test results for the complete test split and for each domain. On the complete split, the mAP@50 is 0.479 for the defaults, 0.497 for random search and 0.500 for PSO. Both tuned arms are significantly better than the defaults (B$-$A $+0.017$ [$+0.006$, $+0.030$] and C$-$A $+0.020$ [$+0.011$, $+0.032$]). PSO and random search, on the other hand, cannot be distinguished (C$-$B $+0.003$ [-0.008, $+0.015$]). The intervals of the individual arms overlap to a large extent. The gains are detected only because the paired bootstrap removes the variation in image difficulty, which is common to all three models. For mAP@50–95 the verdicts are the same as for mAP@50 in all cases. On the validation split random search was ahead of PSO (0.473 versus 0.466), but on the test split the two are tied.

Out of the 8,192 test boxes, 7,413 come from studio images. The studio test set alone, i.e., the 1,046 test images of Generation 1, leads to the same conclusion: B$-$A ($+0.024$) and C$-$A ($+0.019$) are significant and C$-$B (-0.005) is not. The only place where PSO and random search differ significantly is the Indian test set, where all the scores are close to zero (Section IV-E).

Looking at the individual classes on the complete split, PSO has the highest AP@50 on five of the seven classes and random search on BIODEGRADABLE and PLASTIC, while the defaults are not the best on any class. The OTHER class is very weak in all arms (AP@50 of 0.050–0.063), and all of its 132 test boxes are in the real-world images. Precision increases from 0.549 (A) to 0.563 (B) and 0.582 (C), and recall increases from 0.445 (A) to 0.470 for both tuned arms. The median latency on the T4 at 640 px lies between 8.1 and 8.7 ms per image.

E. Domain Gap

When averaged over the three Generation 2 arms, the test mAP@50 drops from about 0.59 on studio images to 0.31 on real-world images and 0.03 on Indian images (Fig. 5). For each arm, the real-world score is approximately half of the studio score.

On the 468 real-world test images from TACO and HITL, none of the pairwise differences is significant (Table VII). The intervals are wide and asymmetric, and the reason is that BIODEGRADABLE has just one box in this set. Depending on whether a resample omits or repeats that image, the AP of this class changes sharply, and this class makes up one seventh of the mean. On that single box the arms obtain an AP@50 of 0.249 (A), 0.020 (B) and 0.003 (C). The intervals of B$-$A and C$-$A are 0.17 to 0.18 wide, so differences of the magnitude seen on studio images (around 0.02) cannot be resolved on this set.

On the 700 DWSD images from India, all three models fail. The mAP@50 is between 0.023 and 0.027, precision is 0.070–0.075 and recall is 0.043–0.047. Only PLASTIC (0.089–0.101) and PAPER (0.028–0.035) have an AP@50 above 0.01. The defaults and PSO are both significantly above random search and are not different from each other. For mAP@50–95, the two significant differences are B$-$A -0.002 [-0.003, -0.001]

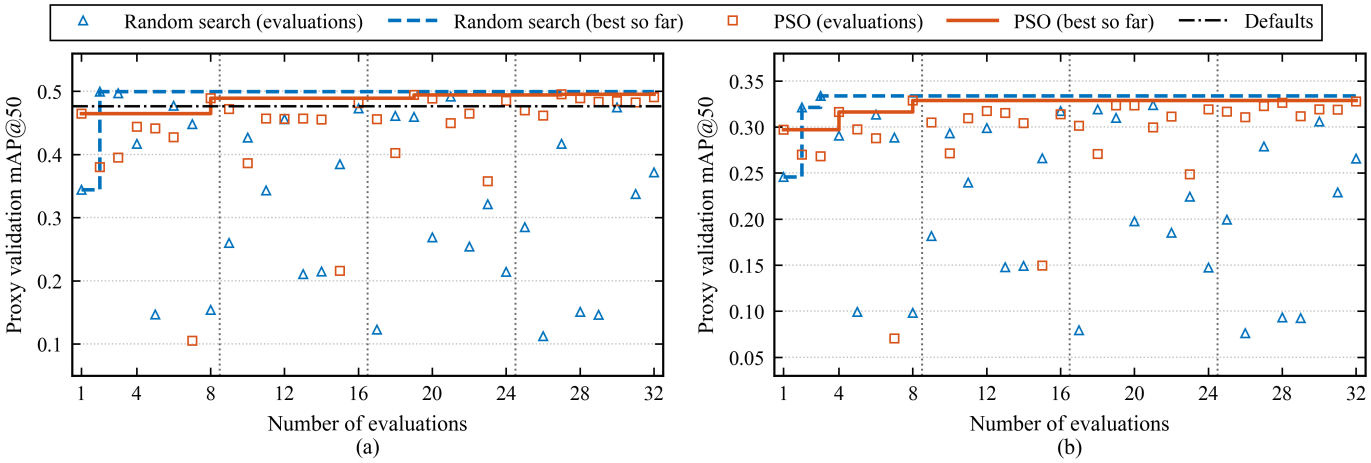


Fig. 3. Convergence of the two searches on the proxy fitness (validation mAP@50 after 12 epochs on 40% of the training split) for (a) Generation 1 and (b) Generation 2. Markers show the individual evaluations and lines show the best score obtained so far. The vertical dotted lines separate the four PSO rounds of eight particles each; random search has no rounds and is plotted in the order of sampling. The dash-dotted line in (a) is the score of the defaults on the same proxy, which was not run for Generation 2. The proxy scores of the two panels are not comparable since the validation sets, classes and image sizes are different.

TABLE VI
GENERATION 1 TEST RESULTS ON THE STUDIO TEST SPLIT (1,046 IMAGES)

Arm	Val mAP@50	Test mAP@50 [95% CI]	Test mAP@50–95 [95% CI]	P	R	ms/img
A: Defaults	0.657	0.676 [0.639, 0.722]	0.474 [0.441, 0.518]	0.747	0.581	31.4
B: Random search	0.643	0.672 [0.634, 0.717]	0.475 [0.441, 0.519]	0.746	0.587	38.3
C: PSO	0.657	0.668 [0.627, 0.717]	0.471 [0.437, 0.516]	0.747	0.586	31.7

Pair	ΔmAP@50 [95% CI]	$P(\Delta > 0)$	ΔmAP@50–95 [95% CI]	$P(\Delta > 0)$	Verdict
B−A	−0.004 [−0.016, +0.009]	0.30	+0.001 [−0.008, +0.010]	0.59	not significant
C−A	−0.008 [−0.021, +0.007]	0.16	−0.003 [−0.012, +0.007]	0.34	not significant
C−B	−0.004 [−0.015, +0.007]	0.24	−0.003 [−0.011, +0.004]	0.19	not significant

Val: best checkpoint on the validation split after 100 epochs. CIs: 1,000 paired bootstrap resamples of the test images. P and R at the confidence of maximum F1. ms/img: median batch-1 latency at 416 px on a laptop CPU (Intel Core Ultra 5 125H). $P(\Delta > 0)$: share of resamples in which the first arm of the pair scored higher. A difference is significant only if its CI excludes 0.

TABLE VII
GENERATION 2 TEST RESULTS PER TEST SET: MAP@50 AND MAP@50–95 [95% CI] PER ARM, AND PAIRWISE DIFFERENCES IN MAP@50

	All 1,514 img., 8,192 boxes	Studio 1,046 img., 7,413 boxes	Real-world 468 img., 779 boxes	India (DWSD) 700 img., 4,502 boxes
Test mAP@50 [95% CI]				
A: Defaults	0.479 [0.453, 0.513]	0.571 [0.529, 0.619]	0.310 [0.278, 0.440]	0.027 [0.024, 0.031]
B: Random search	0.497 [0.471, 0.531]	0.595 [0.557, 0.639]	0.298 [0.271, 0.407]	0.023 [0.021, 0.027]
C: PSO	0.500 [0.474, 0.534]	0.590 [0.550, 0.635]	0.310 [0.279, 0.415]	0.027 [0.024, 0.031]
ΔmAP@50 [95% CI] (bold: significant, CI excludes 0)				
B−A	**+0.017** [+0.006, +0.030]	**+0.024** [+0.009, +0.038]	−0.012 [−0.118, +0.053]	**−0.003** [−0.006, −0.001]
C−A	**+0.020** [+0.011, +0.032]	**+0.019** [+0.005, +0.033]	+0.001 [−0.111, +0.065]	+0.000 [−0.003, +0.003]
C−B	+0.003 [−0.008, +0.015]	−0.005 [−0.018, +0.009]	+0.013 [−0.015, +0.036]	**+0.003** [+0.001, +0.006]
Test mAP@50–95 [95% CI]				
A: Defaults	0.329 [0.309, 0.358]	0.386 [0.356, 0.425]	0.254 [0.224, 0.371]	0.015 [0.013, 0.018]
B: Random search	0.342 [0.322, 0.372]	0.404 [0.374, 0.442]	0.238 [0.213, 0.330]	0.013 [0.012, 0.015]
C: PSO	0.343 [0.323, 0.374]	0.398 [0.367, 0.439]	0.248 [0.220, 0.336]	0.015 [0.013, 0.017]

All test sets were scored once, at 640 px, with 1,000 paired bootstrap resamples per test set. "All" is the whole Generation 2 test split (studio plus real-world); India is DWSD, used for testing only. The mAP@50–95 differences give the same verdict as the mAP@50 differences in every cell. Validation mAP@50: A 0.454, B 0.473, C 0.466.

and C−B +0.002 [+0.000, +0.003]; the lower bound of the latter is 0.0004 before rounding. Since these differences are of a few thousandths on scores that are nearly zero, they have no practical significance.

Four reasons for this failure can be identified from the data (Section III-B). First, no Indian image was used in training, search or model selection. Second, the DWSD images appear to be augmented (rotated, with black corners and color tints)

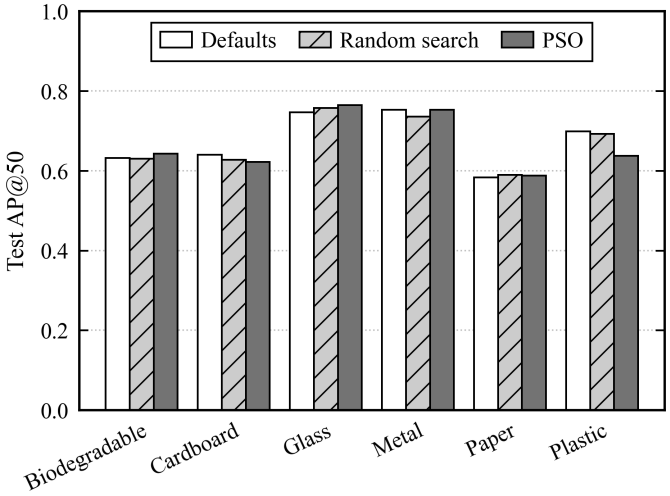


Fig. 4. Per-class test AP@50 of the three Generation 1 models on the studio test split. The number of test boxes as counted by the evaluator is 4,533 for BIODEGRADABLE, 392 for CARDBOARD, 822 for GLASS, 588 for METAL, 527 for PAPER and 553 for PLASTIC (7,415 in total, including 2 zero-size BIODEGRADABLE boxes). PSO is lowest on PLASTIC, and on the other classes the arms are within 0.02 of each other.

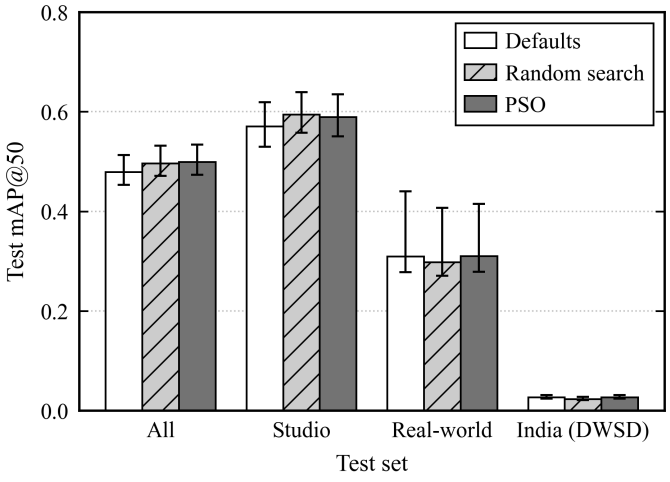


Fig. 5. Test mAP@50 of the three Generation 2 models on each test set. The error bars are 95% paired-bootstrap confidence intervals. "All" (1,514 images) combines the studio (1,046) and real-world (468) test images, and India (DWSD, 700 images) was used for testing only. The real-world intervals are asymmetric because that set has only one BIODEGRADABLE box.

and look unlike any of the training images. Third, the DWSD boxes are derived from mask regions. Touching items of a class are merged into one large box, which overlaps poorly with per-item detections, so that even correct detections are often counted as misses. Fourth, the mapping of mask values to classes was inferred from the images, and six of the fifteen values are uncertain. The India score is therefore a measure of the difference in labeling style as much as of the models. Fig. 6 shows the output of the three models for two studio, two real-world and two DWSD test images. Both real-world images contain eight objects. In the first one each model finds three objects at most, and in the second none. On the DWSD images each model predicts at most one box, whereas the ground truth has 4 and 19 boxes. In the first DWSD image, all three models

TABLE VIII
GENERATION 1 VERSUS GENERATION 2 ON THE SAME TEST IMAGES: TEST MAP@50 [95% CI] ON THE SIX SHARED CLASSES

Model (px)	Real-world (468 img.)	Studio (1,046 img.)
G1 A (416)	0.047 [0.038, 0.080]	0.676 [0.641, 0.719]
G1 B (416)	0.038 [0.031, 0.069]	0.672 [0.636, 0.716]
G1 C (416)	0.096 [0.041, 0.214]	0.669 [0.628, 0.716]
G2 A (640)	0.353 [0.315, 0.504]	0.571 [0.529, 0.619]
G2 B (640)	0.338 [0.304, 0.473]	0.595 [0.557, 0.639]
G2 C (640)	0.352 [0.315, 0.485]	0.590 [0.550, 0.635]
Δ G2$-$G1	$+0.243$ to $+0.315$	-0.105 to -0.074

G1, G2: Generation 1, Generation 2; each model was scored once at its own training image size (px). Real-world: 647 boxes after removing OTHER, with a single BIODEGRADABLE box; studio: 7,413 boxes, no OTHER. CIs: 1,000 paired bootstrap resamples per test set. Δ: range over all nine G2$-$G1 pairs, every one significant; e.g., real-world G2 B$-$G1 C $+0.243$ [$+0.134$, $+0.418$], studio G2 A$-$G1 A -0.105 [-0.125, -0.084]. On the five real-world classes with more than one box, Generation 2 scores 0.239–0.618 AP@50 and Generation 1 0.008–0.117.

draw a box around the region which the ground truth labels as PAPER, but classify it as PLASTIC.

As mentioned before, the two generations have different classes, image sizes and training data, and Tables VI and VII cannot be read as one benchmark. In Table VIII, all six final models are therefore evaluated on the same studio and real-world test images using only the six shared classes (Section III-I). On the real-world images, every Generation 2 arm is significantly better than every Generation 1 arm, with a mAP@50 of 0.338–0.353 against 0.038–0.096, which is roughly four to nine times higher. On the studio images the situation is reversed, and every Generation 1 arm is significantly better than every Generation 2 arm by 0.074 to 0.105. Within a generation, none of the real-world differences is significant (e.g., Generation 1 C$-$A $+0.048$ [-0.020, $+0.163$] and Generation 2 C$-$B $+0.013$ [-0.019, $+0.042$]), and the studio verdicts are the same as in Tables VI and VII. The value of 0.096 for Generation 1 C and its wide interval are mainly caused by the single real-world BIODEGRADABLE box, on which this model has an AP@50 of 0.332. The real-world scores of Generation 2 are higher here than in Table VII (0.353 against 0.310 for Arm A) only because OTHER is excluded from the mean. As a sanity check, Generation 1 reproduces its own studio test result to within 0.0004 mAP@50.

To understand where the Generation 2 models do succeed on real-world photographs, we analyzed the 468 real-world test images one by one. An image is considered fully correct if every ground-truth object is matched by a detection of the same class with IoU ≥ 0.5 and no detection remains unmatched. Matching is greedy in the order of decreasing confidence, with a confidence threshold of 0.25. The models are fully correct on 166 (A), 168 (B) and 170 (C) images, i.e., 35 to 36% of the set, and all three are correct on the same 111 images. Nearly all of these are photographs with a single object. Of the 375 images with one object, the models are fully correct on 164 to 167. Of the 47 images with two objects, they are correct on 1 to 4, and of the 46 images with three or more objects, on none. Fig. 7 shows the most favorable cases, which were selected by a fixed rule. No image with three or more

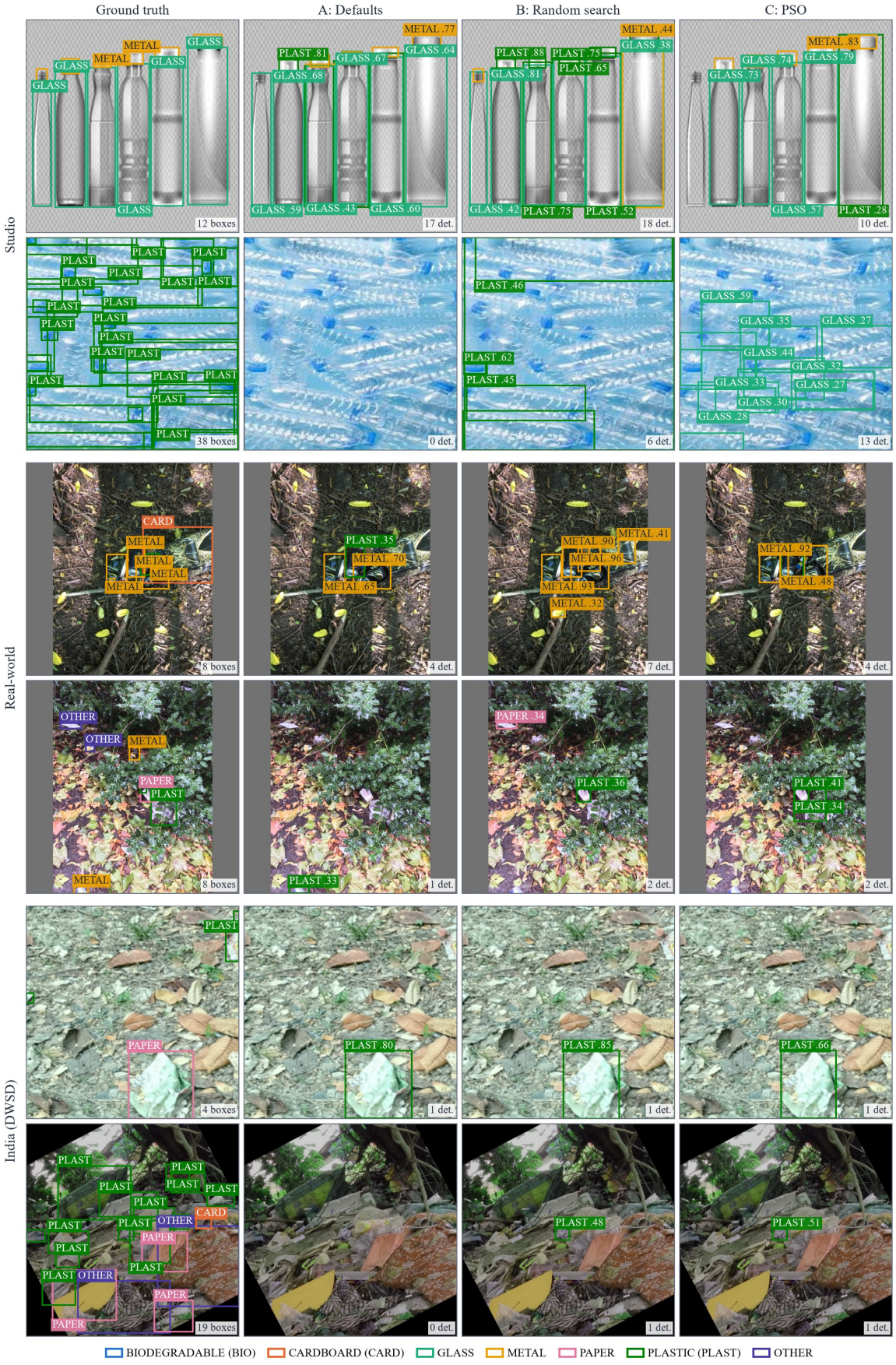


Fig. 6. Detections of the three Generation 2 models on two studio, two real-world and two Indian (DWSD) test images, next to the ground truth. The images were not picked by hand: for each test set, the images having at least three ground-truth boxes were shuffled with seed 0 and the first two are shown. All models were given the same images at 640 px with a confidence threshold of 0.25 and an NMS IoU of 0.7; the real-world rows show the predictions recorded in the per-image analysis of Fig. 7. Each box is marked with its class and, for predictions, its confidence, and a label is omitted if it would cover another one. The number of ground-truth boxes or detections is given in the corner of each panel. These examples are only illustrative; for the quantitative results see Table VII and Fig. 5. The DWSD ground-truth boxes come from mask regions, so touching items of one class can merge into one box.

TABLE IX
RECORDED GPU TIME PER RUN (HOURS, ONE TESLA T4)

Run	Generation 1	Generation 2
Proxy-validity check (16 runs)	4.36	not run
PSO search (32 proxy trainings)	1.91	2.54
Random search (32 proxy trainings)	2.12	2.30
Arm A full training (100 epochs)	1.18	1.56
Arm B full training (100 epochs)	1.15	2.07
Arm C full training (100 epochs)	1.45	1.60
Total recorded	12.18	10.06

Sums of the recorded training wall-clock times. The Generation 2 random search and Arm C training were resumed after a session froze (Arm C after epoch 55); their idle time, about 24 GPU-hours in total, is not included.

objects is fully correct for all three models, and only one image with two objects is. The rule therefore falls back on the two images with at least two objects that are labeled fully correctly by at least two models. One of them is a close-up on a plain background, which is close to studio conditions. In short, the models can deal with an isolated item in a real scene but not with a cluttered scene containing several items.

F. Compute Cost

The GPU time recorded for each run on the Tesla T4 is given in Table IX. Only the training time is counted; setup, copying of data and idle time are excluded. When the search is included, each tuned arm costs about 2.7 to 2.8 times the GPU time of Arm A, which requires no search. The budget was equal in terms of evaluations but not in terms of time. A proxy training took 214.3 s on average for PSO and 238.7 s for random search in Generation 1, and 285.5 s and 259.0 s in Generation 2. The most expensive item in Generation 1 was the proxy-validity check, in which the eight 40-epoch medium runs accounted for 3.84 of the 4.36 hours. In Generation 2, about 24 additional GPU-hours were wasted when two sessions froze and remained idle until the 12-hour limit of the platform was reached; these hours are not part of the totals. The one-time test evaluation took 283.1 s on the laptop CPU in Generation 1 and 220.6 s on the T4 for the four test sets of Generation 2.

V. DISCUSSION

A. Why PSO Did Not Beat Random Search

We believe there are several reasons for this outcome. In Generation 1, the default values appear to lie in a wide and flat region of good settings. They scored higher than most of the proxy evaluations of both searches and were behind the two winners by only 0.019 (PSO) and 0.023 (random search), as reported in Section IV-B. In the proxy check they were second out of eight at both training lengths, and after 40 epochs the four best settings were within 0.026 of each other (Table V).

Inside such a region the proxy is able to distinguish poor settings from good ones ($\rho = 0.976$), but it cannot reliably identify the best one. The setting that won on the proxy in Generation 1, that of random search, actually gave the lowest validation score after 100 epochs (Table VI). A short proxy may favor settings that learn quickly. In addition, taking the maximum over 32 single-run scores tends to select a score that is high partly by chance. Neither of these effects was measured by us.

The budget was small as well. There were 32 evaluations for six dimensions, and PSO used eight of them for its random initial swarm. Bergstra and Bengio [10] showed that for most datasets only a few hyperparameters really matter, and that random search finds models that are as good as or better than those of grid search for the same budget. If one or two of our six dimensions dominate, then 32 independent samples already cover them quite densely and little is left for a guided search to exploit. The fact that all four search winners have an initial learning rate below the default is consistent with this explanation, although we did not measure the importance of each dimension.

The swarm also converged early. Its mean proxy score became higher than that of random search, but its best score did not (Section IV-B). In Generation 2, the best particle was the last one of the random initial swarm (Fig. 3). In effect, PSO was offering the best of eight random samples against the best of 32. With only three update steps, a global-best swarm gathers its later evaluations around the best point of round 1. This increases the mean but not the maximum, and only the maximum is used.

It is less clear why tuning helped in Generation 2 alone. Our hypothesis is that the new data (real-world photographs, 640 px images and a seventh class) shifted the good region away from the defaults, so that any reasonable search could gain about 0.02 mAP@50 on the mixed and studio test sets. This could not be verified because there is no proxy run of the defaults for Generation 2. As every arm was trained only once, some of the gain may also be training noise. Repeating the proxy check and the full trainings for Generation 2 would help to separate the two explanations (Section VI-A).

B. Implications for Tuning Claims in Waste AI

According to Gap 3 (Section II-E), metaheuristic tuning in this area is compared with other metaheuristics or with nothing, and single test values are reported. Our results illustrate what can go wrong in that case. If PSO had been compared with the defaults alone, its significant gain of $+0.020$ mAP@50 in Generation 2 would have been attributed to the swarm. Random search, however, gained $+0.017$ with the same budget, and the two tuned arms are indistinguishable (Table VII). The gain is therefore a result of searching as such and not of the swarm. Single values would have been misleading too. In Generation 1 the point estimates place the defaults first and PSO last, and in Generation 2 the validation scores place random search ahead of PSO, but neither order is significant on the test set. In the opposite direction, the significant $+0.003$ of PSO over random search on the India set, where scores are about 0.02, shows that a significant difference also has to be large enough to matter in practice.

On this basis, we suggest the following minimum protocol for any claim that a metaheuristic tunes a waste model better: (1) include the defaults and random search with the same number of evaluations, and report the time taken (Section IV-F);



Fig. 7. Success cases on real-world test images. These are the most favorable examples and are not representative of the whole set. Rows 1–2: the only real-world test images with at least two objects that at least two of the three Generation 2 models label fully correctly (every object found with the right class, no false detection); no image with three or more objects qualifies, and random search (B) misses one object in the first. Rows 3–4: the first two of the 110 single-object images that all three models label fully correctly, in a random order with seed 0. Confidence threshold 0.25, NMS IoU 0.7, 640 px. For scale, each model is fully correct on 166–170 of the 468 real-world test images, on 164–167 of the 375 single-object images and on none of the 46 images with three or more objects.

(2) validate any cheap proxy against longer training before the search, using a threshold decided beforehand; (3) select the final setting of each arm on validation data and evaluate the test split once; (4) report paired confidence intervals for the differences between arms on each test domain, and judge them against a difference that is relevant in practice; and (5) repeat the trainings and searches with several seeds. SwarmSort satisfies all of these except the fifth. For equal-budget comparisons such as the one by Sayed et al. [9], adding a random-search arm would cost no more than one additional competitor.

C. Real-World and Indian Deployment

For a smart bin or a municipal service, the data are far more important than the optimizer. Every arm loses about half of its studio score on real-world photographs and drops to 0.023–0.027 mAP@50 on the Indian ones (Section IV-E). The largest effect of tuning, about 0.02, is less than one tenth of the drop from studio to real-world images (about 0.28), and it was not observed on real-world photographs at all. The strongest evidence comes from Table VIII, where both generations are evaluated on the same images. Real-world training data increased the real-world mAP@50 from 0.038–0.096 in Generation 1 to 0.338–0.353 in Generation 2, with every Generation 2 arm significantly above every Generation 1 arm. Tuning, in contrast, made no significant difference on real-world images in either generation.

This improvement came with a significant loss of 0.074 to 0.105 on studio images. Three causes are likely, and our runs cannot separate them. Generation 2 retained only 41% of

the studio training images of Generation 1. The same small YOLOv8n now has to share its capacity with real-world scenes and a seventh class. Finally, Generation 2 was trained at 640 px instead of 416 px and its checkpoint was selected on mixed validation data. We conclude that local training data will be of more help than a better search, and this is why Indian data are placed first in our future work (Section VI-B). Clutter is the second difficulty. The models label all objects correctly in about 44% of the real-world photographs that contain one object, but in none of the 46 photographs with three or more objects (Fig. 7). A bin that photographs one item at a time is thus a realistic goal at present, whereas sorting a mixed pile is not.

Since the merged, mask-derived boxes of DWSD penalize correct per-item detections (Section IV-E), a test set needs to be labeled in the same manner as the training data before its score can be interpreted as accuracy. Relabeling a part of DWSD item by item would make it possible to separate the labeling style from the model error.

The Indian rules also have a bearing on the classes. Rule 15(h) of the Solid Waste Management Rules, 2016 [1] prescribes green, white and black bins for bio-degradable, recyclable and other waste. BIODEGRADABLE corresponds to the green bin, and the material classes correspond to the white bin to the extent that their items are recyclable. The black bin corresponds to OTHER, which is the weakest class of every arm (AP@50 of 0.050 to 0.063). A bin built on these models would be least reliable exactly there.

D. Limitations and Threats to Validity

This study has a number of limitations. Each arm was trained once (seed 42) and each search was run once (seed 42 for PSO and seed 123 for random search). The confidence intervals therefore cover only the sampling of test images, and the small differences between arms could change with another seed. We did not compute p-values or correct for multiple comparisons, although Generation 2 involves twelve pairwise comparisons per metric (three pairs on four test sets). Per-class AP has no interval, and the single BIODEGRADABLE box in the real-world test set makes the intervals of that set wide and asymmetric.

The proxy check was based on eight settings, was done for Generation 1 only and used a 40-epoch reference instead of a 100-epoch one, so the validity of the proxy in Generation 2 is assumed. The proxy evaluated the weights of the last epoch, while the final models used the best validation checkpoint (`best.pt`). The Arm C training of Generation 2 was resumed after epoch 55 because of a frozen session. It kept the same weights and optimizer state but is not bit-identical to an uninterrupted run.

For DWSD, the class mapping was inferred from the masks with six of the fifteen mask values uncertain, and the boxes are connected regions and not individual items. Only six training hyperparameters were searched, and the budget was equal in evaluations but not in time. We did not run any external baseline, such as the pipeline of Majchrowska et al. [6], on our data, so our scores cannot be ranked against earlier work. Lastly, the literature review was targeted and not systematic (Section II-A), and the gaps in Section II-E hold only for the literature that we reviewed.

VI. Future Work: Possible Improvements

The improvements proposed in this section correspond either to a limitation listed in Section V-D or to one of the measured results. They are ranked in Table X according to the benefit we expect for the effort involved.

A. The Optimization Study

With three training seeds per arm, the intervals would also include training noise, and with several search seeds per method it would become clear whether the PSO result is repeatable. The small proxy differences among the top settings disappeared after full training. A multi-fidelity search such as successive halving or Hyperband [38] could handle this by promoting the best candidates to longer runs before a winner is selected. Since the swarm converged early, and in Generation 2 did not improve at all after round 1, PSO variants that maintain exploration are worth trying. Examples are the constriction factor of Clerc and Kennedy [37], a ring topology, an adaptive inertia weight, or simply more particles and rounds.

Other optimizers can be added as further arms against the same baselines with 32 evaluations, e.g., a genetic or evolutionary algorithm [45], differential evolution [46] and Bayesian optimization such as the tree-structured Parzen estimator of Optuna [47]. This also answers the question of why PSO was chosen and not, for instance, the beluga whale optimizer of Sayed et al. [9]. The protocol does not depend on the optimizer, and its value lies in confronting every candidate with the same baselines. A larger search space that includes color augmentation, translation, loss gains and image size would show whether six settings were too few to beat the defaults. A domain-aware fitness computed on real-world validation images could address the fact that the tuning gains did not reach real-world photographs. The proxy check should also be repeated for Generation 2 with a 100-epoch reference and a run of the defaults, which would test the hypothesis of Section V-A. On the statistical side, Holm's correction, intervals for per-class AP and stored bootstrap resamples would strengthen the analysis.

B. Data and the Indian Domain

The most important next step is to obtain Indian training data with per-item boxes, either by relabeling a part of DWSD or through a new collection with a held-out test portion. This would address the failure on DWSD as well as the labeling mismatch that is partly responsible for it. Because the real-world data of Generation 2 reduced the studio accuracy (Section V-C), new data should be added through domain-aware training, for example by retaining all studio images or by using a larger model. The weak OTHER class requires more examples, or it should be divided into clearer classes or merged with existing ones. Clutter-oriented augmentations such as copy-paste [48], smaller object scales, blur and lighting

changes are aimed at the small, overlapping and partly hidden items seen in real-world photographs. The classes could also be aligned with the three streams required by the SWM Rules 2016, namely bio-degradable, non-bio-degradable and domestic hazardous [1], so that every detection corresponds directly to a bin.

C. The Model

Training larger or newer detectors (YOLOv8s, YOLOv8m, YOLO11 [49]) with the same settings would reveal how much of the real-world gap is caused by limited model capacity, though at the expense of speed on edge devices. Sliced inference on overlapping tiles [50] is suited to wide photographs with many small items. Confidence calibration, e.g., temperature scaling [51], would make a score of 0.5 correspond to roughly 50% correct detections, which is important when a bin acts on a threshold.

D. Combining the Three Models

According to the point estimates, no single arm is the best on every Generation 2 test set (Table VII). On real-world photographs the three mAP@50 values are within 0.012 of each other, but the arms differ more in the details. Precision at the maximum-F1 confidence varies from 0.366 (B) to 0.556 (C), the AP@50 of PAPER varies from 0.239 (A) to 0.330 (C), and random search is the best on GLASS (0.296). These values have no intervals and the agreement between the models was not measured. Still, they suggest that the models make partly different errors, and a combination of the three arms, which requires no additional training, might perform better than any one of them.

Three ways of combining them can be compared. The first is box fusion, in which the boxes of A, B and C are pooled and the overlapping ones are merged by weighted boxes fusion [52] or non-maximum suppression. The second is majority voting, where a box is kept only when at least two models find it (IoU ≥ 0.5). This should raise precision, and boxes found by a single model can be flagged as uncertain rather than thrown away. The third is weighted voting according to the validation mAP@50 of each model. Test scores must not be used as weights, as that would leak the test split. From the soft-computing point of view, a search produces a whole population of good settings (the final swarm and the best random samples). Training the top-k settings and combining them would turn the swarm into an ensemble generator. The converged swarm may not be diverse enough, so the best random samples should be mixed in. Every ensemble should be compared with the best single arm, selected on validation data, using the same paired bootstrap on each test set. The price is inference time, which becomes about three times longer: roughly 100 ms per image on the laptop CPU, as against 31 to 38 ms for a single Generation 1 model.

E. Deployment

The single confidence threshold could be replaced by per-class thresholds chosen on validation data, because real-world precision is low and AP differs widely between classes. Our latency figures come only from a laptop CPU and a cloud GPU. The models should therefore be exported to ONNX or TFLite and timed on a Raspberry Pi, which is the kind of hardware used by Alshammari et al. [15] for their smart-bin classifier. As actual sorting is a continuous process, the models should also be tested on live video.

TABLE X
FUTURE-WORK ROADMAP, ORDERED BY EXPECTED VALUE FOR EFFORT

Priority	Improvement	Addresses	Effort
1	Indian training data with per-item boxes	DWSD failure (0.023–0.027 mAP@50)	Large
2	Ensemble of A, B and C (fusion or voting)	Arms differ per class and test set; no arm wins everywhere	Small (no training)
3	Three training seeds per arm, several search seeds	Tuning effect vs. training noise	Medium
4	Domain-aware fitness (real-world validation)	Tuning gains did not carry to real-world photos	Medium
5	Multi-fidelity search (Hyperband)	Proxy gaps vanish at 100 epochs	Medium
6	GA, DE and Bayesian optimization at the same budget	"Why PSO?"	Medium
7	Fix the OTHER class	AP@50 0.050–0.063	Medium
8	Larger detector; sliced inference	Small items, look-alike materials	Medium
9	Edge export and latency on a Raspberry Pi	Real smart-bin hardware	Medium
10	Swarm as an ensemble generator (top-k settings)	Uses the whole PSO population	Medium

Effort: Small = hours, no GPU training; Medium = about a day with some GPU time; Large = several days or a new data pipeline. GA = genetic algorithm; DE = differential evolution.

VII. CONCLUSION

In this paper, we presented SwarmSort, a comparison of particle swarm optimization, random search and the default settings for tuning a lightweight waste detector under an equal budget of 32 proxy trainings.

With regard to RQ1, PSO did not outperform random search in any meaningful way. In Generation 1, which used studio data, the three arms were statistically tied (0.676, 0.672 and 0.668 mAP@50). In Generation 2, both tuned arms were better than the defaults on the mixed test set (random search by $+0.017$ and PSO by $+0.020$), but PSO and random search could not be distinguished on the mixed, studio and real-world test sets. Their only significant difference, $+0.003$ on the India set, occurs at scores of about 0.02 and has no practical value. With regard to RQ2, the short proxy ranked the settings like a longer training ($\rho = 0.976$, $n = 8$, Generation 1 only). The small differences it showed between the best settings, however, disappeared after full training. With regard to RQ3, the answer does depend on the domain. The gains from tuning did not carry over to real-world photographs (0.298 to 0.310), and all arms failed on the Indian DWSD photographs (0.023 to 0.027), which is partly due to the merged boxes of DWSD.

Real-world training data, on the other hand, raised real-world mAP@50 from 0.038–0.096 in Generation 1 to 0.338–0.353 in Generation 2 on the same images, with a loss of 0.074 to 0.105 on studio images.

Two conclusions can be drawn from this work. First, a claim that a metaheuristic tunes a model better should be supported by the defaults and random search at an equal budget, and by confidence intervals on a test split that is used once. Had random search been left out, the Generation 2 gain of PSO would have been attributed to the swarm. Second, for waste detection in real scenes, and particularly in Indian scenes, the training data are much more important than the optimizer. As discussed in Section VI, our immediate future work is to collect Indian training data with per-item boxes and to build an ensemble of the three models.

Data and Code Availability

All the data used in this study are publicly available. Generation 1 is based on the Kaggle dataset "Garbage Detection – 6 Waste Categories" [40] (CC BY 4.0). For Generation 2, TACO [17] (CC BY 4.0) and the Humans in the Loop Recycling dataset [41] (CC0) were added. The Indian test set was prepared from DWSD [13] (CC BY 4.0), which can be downloaded from Mendeley Data at https://doi.org/10.17632/gr99ny6b8p.1. The code, the split file, the search logs and the result files are provided in the project repository at https://github.com/CrossAnushka/Particle-Swarm-Optimization-for-Waste-Detection, along with the dataset-build configuration that maps each source to the shared classes. The trained weights of all six models (three arms for each of the two generations) are also included in the repository under `app/models/` (Generation 1) and `app/models/gen2/` (Generation 2).

References

[1] Ministry of Environment, Forest and Climate Change, Government of India, "Solid waste management rules, 2016," Notification S.O. 1357(E), Gazette of India, Extraordinary, Part II, Section 3, Sub-section (ii), No. 861, Apr. 2016. [Online]. Available: https://cpcb.gov.in/uploads/MSW/SWM_2016.pdf

[2] M. Abdallah, M. Abu Talib, S. Feroz, Q. Nasir, H. Abdalla, and B. Mahfood, "Artificial intelligence applications in solid waste management: A systematic research review," *Waste Management*, vol. 109, pp. 231–246, 2020.

[3] B. Fang, J. Yu, Z. Chen, A. I. Osman, M. Farghali, I. Ihara, E. H. Hamza, D. W. Rooney, and P.-S. Yap, "Artificial intelligence for waste management in smart cities: a review," *Environmental Chemistry Letters*, vol. 21, no. 4, pp. 1959–1989, 2023.

[4] M. Yang and G. Thung, "Classification of trash for recyclability status," Stanford University, CS229 Project Report, 2016. [Online]. Available: https://cs229.stanford.edu/proj2016/report/ThungYang-ClassificationOfTrashForRecyclabilityStatus-report.pdf

[5] V. Kaya, "Classification of waste materials with a smart garbage system for sustainable development: a novel model," *Frontiers in Environmental Science*, vol. 11, p. 1228732, 2023.

[6] S. Majchrowska, A. Mikołajczyk, M. Ferlin, Z. Klawikowska, M. A. Plantykow, A. Kwasigroch, and K. Majek, "Deep learning-based waste detection in natural and urban environments," *Waste Management*, vol. 138, pp. 274–284, 2022.

[7] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4. Perth, WA, Australia: IEEE, 1995, pp. 1942–1948.

[8] K. Aguerchi, Y. Jabrane, M. Habba, and A. Hajjam El Hassani, "A CNN hyperparameters optimization based on particle swarm optimization for mammography breast cancer classification," *Journal of Imaging*, vol. 10, no. 2, p. 30, 2024.

[9] G. I. Sayed, M. Abd Elfattah, A. Darwish, and A. E. Hassanien, "Intelligent and sustainable waste classification model based on multi-objective beluga whale optimization and deep learning," *Environmental Science and Pollution Research*, vol. 31, no. 21, pp. 31 492–31 510, 2024.

[10] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012. [Online]. Available: https://jmlr.org/papers/v13/bergstra12a.html

[11] W. Lu and J. Chen, "Computer vision for solid waste sorting: A critical review of academic research," *Waste Management*, vol. 142, pp. 29–43, 2022.

[12] M. Córdova, A. Pinto, C. C. Hellevik, S. A.-A. Alaliyat, I. A. Hameed, H. Pedrini, and R. d. S. Torres, "Litter detection with deep learning: A comparative study," *Sensors*, vol. 22, no. 2, p. 548, 2022.

[13] A. Ali, S. Acharjee, M. M. Sk, S. Z. Alharthi, S. S. Chaudhuri, and A. Akhunzada, "DWSD: Dense waste segmentation dataset," *Data in Brief*, vol. 59, p. 111340, 2025.

[14] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Software, version 8.0.0, AGPL-3.0 licence, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[15] A. Alshammari, S. Alkhabbaz, R. Almutairi, H. Alhabad, S. Alhamdan, and I. Boulkaibet, "Smart waste classification system using deep learning for automated recycling," in *2025 6th International Conference on Bio-engineering for Smart Technologies (BioSMART)*. Paris, France: IEEE, 2025, pp. 1–6.

[16] N. V. Kumsetty, A. B. Nekkare, S. Kamath S., and A. Kumar M., "TrashBox: Trash detection and classification using quantum transfer learning," in *2022 31st Conference of Open Innovations Association (FRUCT)*, 2022, pp. 125–130.

[17] P. F. Proença and P. Simões, "TACO: Trash annotations in context for litter detection," arXiv preprint arXiv:2003.06975, 2020.

[18] D. Bashkirova, M. Abdelfattah, Z. Zhu, J. Akl, F. Alladkani, P. Hu, V. Ablavsky, B. Calli, S. A. Bargal, and K. Saenko, "ZeroWaste dataset: Towards deformable object segmentation in cluttered scenes," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 21 115–21 125.

[19] D. Yudin, N. Zakharenko, A. Smetanin, R. Filonov, M. Kichik, V. Kuznetsov, D. Larichev, E. Gudov, S. Budennyy, and A. Panov, "Hierarchical waste detection with weakly supervised segmentation in images from recycling plants," *Engineering Applications of Artificial Intelligence*, vol. 128, p. 107542, 2024.

[20] M. Kraft, M. Piechocki, B. Ptak, and K. Walas, "Autonomous, onboard vision-based trash and litter detection in low altitude aerial images collected by an unmanned aerial vehicle," *Remote Sensing*, vol. 13, no. 5, p. 965, 2021.

[21] J. Hong, M. Fulton, and J. Sattar, "TrashCan: A semantically-segmented dataset towards visual detection of marine debris," arXiv preprint arXiv:2007.08097, 2020.

[22] Y. Cheng, J. Zhu, M. Jiang, J. Fu, C. Pang, P. Wang, K. Sankaran, O. Onabola, Y. Liu, D. Liu, and Y. Bengio, "FloW: A dataset and benchmark for floating waste detection in inland waters," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10 933–10 942.

[23] T. Wang, Y. Cai, L. Liang, and D. Ye, "A multi-level approach to waste object segmentation," *Sensors*, vol. 20, no. 14, p. 3816, 2020.

[24] M. Fulton, J. Hong, M. J. Islam, and J. Sattar, "Robotic detection of marine litter using deep visual detection models," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 5752–5758.

[25] D. O. Melinte, A.-M. Travediu, and D. N. Dumitriu, "Deep convolutional neural networks object detector for real-time waste identification," *Applied Sciences*, vol. 10, no. 20, p. 7301, 2020.

[26] Y.-H. Liao and J.-G. Juang, "Real-time UAV trash monitoring system," *Applied Sciences*, vol. 12, no. 4, p. 1838, 2022.

[27] M. Bartolo, K. Makantasis, and D. Seychell, "Learning using privileged information for litter detection," arXiv preprint arXiv:2508.04124, 2025.

[28] A. Sánchez-Ferrer, J. J. Valero-Mas, A. J. Gallego, and J. Calvo-Zaragoza, "An experimental study on marine debris location and recognition using object detection," *Pattern Recognition Letters*, vol. 168, pp. 154–161, 2023.

[29] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *2016 IEEE Conference on*

Computer Vision and Pattern Recognition (CVPR). Las Vegas, NV, USA: IEEE, 2016, pp. 779–788.

[30] S. Kumar, D. Yadav, H. Gupta, O. P. Verma, I. A. Ansari, and C. W. Ahn, "A novel YOLOv3 algorithm-based deep learning approach for waste segregation: Towards smart waste management," *Electronics*, vol. 10, no. 1, p. 14, 2021.

[31] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, ser. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2014, pp. 740–755.

[32] G. Mittal, K. B. Yagnik, M. Garg, and N. C. Krishnan, "SpotGarbage: Smartphone app to detect garbage using deep learning," in *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp '16)*, 2016, pp. 940–945.

[33] H. Panwar, P. K. Gupta, M. K. Siddiqui, R. Morales-Menendez, P. Bhardwaj, S. Sharma, and I. H. Sarker, "AquaVision: Automating the detection of waste in water bodies using deep transfer learning," *Case Studies in Chemical and Environmental Engineering*, vol. 2, p. 100026, 2020.

[34] M. R. Isalm, N. B. Mahabub, M. J. A. Rafi, P. K. Roy, T. Roy, M. T. Islam, and M. A. Razzak, "UGV-NBWASTE: An oriented dataset for non-biodegradable waste in Bangladesh," *Data in Brief*, vol. 60, p. 111559, 2025.

[35] S. R. Mandhati, N. L. Deshapriya, C. L. Mendis, K. Gunasekara, F. Yrle, A. Chaksan, and S. Sanjeev, "pLitterStreet: Street level plastic litter detection and mapping," arXiv preprint arXiv:2401.14719, 2024.

[36] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence*. Anchorage, AK, USA: IEEE, 1998, pp. 69–73.

[37] M. Clerc and J. Kennedy, "The particle swarm - explosion, stability, and convergence in a multidimensional complex space," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 1, pp. 58–73, 2002.

[38] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018. [Online]. Available: https://jmlr.org/papers/v18/16-558.html

[39] W.-L. Mao, W.-C. Chen, C.-T. Wang, and Y.-H. Lin, "Recycling waste classification using optimized convolutional neural network," *Resources, Conservation and Recycling*, vol. 164, p. 105132, 2021.

[40] viswaprakash1990, "Garbage detection – 6 waste categories," Kaggle dataset, CC BY 4.0, accessed: Sep. 30, 2026. [Online]. Available: https://www.kaggle.com/datasets/viswaprakash1990/garbage-detection

[41] Humans in the Loop, "Recycling dataset," Kaggle dataset, CC0, accessed: Oct. 7, 2026. [Online]. Available: https://www.kaggle.com/datasets/humansintheloop/recycling-dataset

[42] M. D. McKay, R. J. Beckman, and W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[43] C. Spearman, "The proof and measurement of association between two things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904.

[44] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Springer US, 1993.

[45] S. R. Young, D. C. Rose, T. P. Karnowski, S.-H. Lim, and R. M. Patton, "Optimizing deep learning hyper-parameters through an evolutionary algorithm," in *Proceedings of the Workshop on Machine Learning in High-Performance Computing Environments (MLHPC)*, 2015, pp. 1–5.

[46] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, pp. 341–359, 1997.

[47] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.

[48] G. Ghiasi, Y. Cui, A. Srinivas, R. Qian, T.-Y. Lin, E. D. Cubuk, Q. V. Le, and B. Zoph, "Simple copy-paste is a strong data augmentation method for instance segmentation," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 2917–2927.

[49] G. Jocher and J. Qiu, "Ultralytics YOLO11," Software, version 11.0.0, AGPL-3.0 licence, 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[50] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *2022 IEEE International Conference on Image Processing (ICIP)*. Bordeaux, France: IEEE, 2022, pp. 966–970.

[51] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[52] R. Solovyev, W. Wang, and T. Gabruseva, "Weighted boxes fusion: Ensembling boxes from different object detection models," *Image and Vision Computing*, vol. 107, p. 104117, 2021.